

DLDCALab 3: Performance

Week 3

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to optimize code by exploiting the power of SIMD instructions. In a nutshell, SIMD instructions allow you to *vectorize* your code. They exploit parallelism inherent in certain workloads to boost performance.

2 Necessary tools

We shall be using gcc and g++ in order to compile C and C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ gcc
gcc: fatal error: no input files
compilation terminated.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has access to SIMD 256-bit registers. For this, run the following:

```
$ lscpu | grep -i avx2
... avx2 ....
```

3 Structure of submission/templates

```
your-roll-no/
|- common
  |- matrix.hpp
|- task1/
  |- Makefile
  |- main.cpp
  |- naive.cpp
  |- simd.cpp (TODO)
  |- vector.hpp
|- task2/
  |- main.cpp
```

```
| - Makefile
| - naive.cpp (TODO)
| - optim.cpp
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of two subtasks. These tasks are expected to be submitted in-lab. The first task is intended to get you comfortable with using SIMD instructions and thinking about how to vectorize your code. The second task explores how seemingly complicated operations can be vectorized if they are independent of each other.

Task 1: What is this? DSA?

The primary candidate for *vectorization* is, of course, a *vector*. A vector can be thought of as an arrow pointing from the origin to a point in an n -dimensional space. In CS, it is simply represented as n double precision numbers which are contiguously arranged in memory. The vector class we have implemented has two methods:

1. `dimension`: Takes in no arguments and returns the value of n , if the vector is n dimensional, it is simply the number of doubles in the vector
2. `element`: Takes in an argument, the index i (starting from 0) and returns the address of the i^{th} double in the vector. Since it returns the address, it can be used for both obtaining the value of the i^{th} double and for modifying the value of the i^{th} double

We have given you the naive implementations of the following operations on vectors:

1. `vecAdd(vector* v1, vector* v2)`: Performs element-wise addition of $v1$ and $v2$ and stores the result in a new vector, which is then returned
2. `vecSub(vector* v1, vector* v2)`: Performs element-wise subtraction of $v1$ and $v2$ ($v1[i] - v2[i]$, for all i) and stores the result in a new vector, which is then returned
3. `vecElementMul(vector* v1, vector* v2)`: Performs element-wise multiplication of $v1$ and $v2$ and stores the result in a new vector, which is then returned
4. `vecElementDiv(vector* v1, vector* v2)`: Performs element-wise division of $v1$ and $v2$ ($v1[i]/v2[i]$, for all i) and stores the result in a new vector, which is then returned
5. `dotProduct(vector* v1, vector* v2)`: Performs

$$\sum_{i=0}^{i=v1.dimension()} v1[i] * v2[i]$$

and returns the value

Your job is to use SIMD instructions (cheatsheet in Section5) to speed up the code. Here is what an expected output would look like:

```

$ make
$ ./simd
Initialized vectors. Testing add
Duration for add: 348
Addition done, testing subtraction
Duration for subtraction: 200
Subtraction done, testing multiplication
Duration for multiplication: 209
Multiplication done, testing division
Duration for division: 199
Division done, testing dot product
Duration for dot product: 106
Dot Product done
All tests done!

```

Common

We have given you an implementation of a matrix class, along similar lines as the *vector* class. The matrix class has the following properties:

1. **Size:** It returns a pair of two 64-bit unsigned integers, representing the number of rows and the number of columns in the matrix respectively
2. **Element:** It takes in a pair of integers, say (i, j) as arguments and returns the address of the element of the matrix in the i^{th} row and j^{th} column.

When a matrix is initialized, the dimensions have to be specified, and it is initialized with all its elements being 0.

Task 2: Slow Start

We have given you the naive implementations of the following operations on matrix:

1. **transpose(matrix* A):** Returns a pointer to a new matrix, B such that

$$\forall i \forall j, B[i, j] = A[j, i]$$

2. **add(matrix* a, matrix* b):** Performs element-wise addition of a and b and returns a pointer to a new matrix c such that

$$\forall i \forall j, c[i, j] = a[i, j] + b[i, j]$$

3. **sub(matrix* a, matrix* b):** Performs element-wise subtraction of a and b and returns a pointer to a new matrix c such that

$$\forall i \forall j, c[i, j] = a[i, j] - b[i, j]$$

4. **matmul(matrix* a, matrix* b):** Performs matrix multiplication of a and b and returns a pointer to a new matrix c such that

$$\forall i \forall j, c[i, j] = \sum_k a[i, k] * b[k, j]$$

Your job is to use SIMD instructions (cheatsheet in Section5) to speed up the code. Here is what an expected output would look like:

```

$ make
g++ -O0 -mfma -mavx -o optim optim.cpp main.cpp

```

```

$ ./optim
Transpose Time: 5.22635 ms
Addition Time: 4.00598 ms
Subtraction Time: 3.79201 ms
Multiplication Time: 1204.45 ms

```

5 SIMD Instructions Cheatsheet

Instruction	Arguments	Return Value	Effect
_mm256_setzero_pd	None	__m256d	Returns 256 bits which are all zero
_mm256_loadu_pd	mem	__m256d	Loads 256 bits from mem, interprets them as doubles, and returns them
_mm256_storeu_pd	mem, __m256d	None	Stores 256 bits from the register into mem as doubles
_mm256_add_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is the sum of the corresponding input doubles
_mm256_sub_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is first double minus second double
_mm256_mul_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is the product of the input doubles
_mm256_div_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is first double divided by second double
_mm256_fmadd_pd	__m256d, __m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is (first double * second double) + third double
_mm256_max_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is maximum between first double and second double
_mm256_min_pd	__m256d, __m256d	__m256d	Returns 4 doubles packed into 256 bits, each is minimum between first double and second double
_mm256_set1_pd	double	__m256d	Returns 4 doubles packed into 256 bits, each is having the value of the input double

Table 1: Common SIMD instructions